# Context-Aware Vocabulary Pruning for LLM Inference Acceleration: A Dual-Encoder Retrieval Approach

Working draft — MPS results complete; GPU benchmarks pending

*March 2026*


**Abstract**

Autoregressive decoding in large language models is dominated by two costs at each token step: KV-cache attention and the `lm_head` vocabulary projection — a matrix multiply that reads the full $[V \times d]$ weight matrix from GPU memory on every step. At batch size 1, this operation is **memory-bandwidth-bound**: the GPU's arithmetic units sit idle while 525 MB of weights are streamed from VRAM per token. We address this bottleneck through **dynamic vocabulary pruning**, restricting the `lm_head` projection to a context-dependent shortlist of $K$ candidate tokens at each step, reducing memory traffic by up to 99.6%.

We propose a **Dual-Encoder Router** — a lightweight MLP (0.06% of model parameters) trained with Multiple Negatives Ranking Loss to map the model's hidden state to a query vector, then retrieve the top-$K$ token candidates by nearest-neighbour search over a pre-indexed corpus of sentence completions. We benchmark four routing strategies against a full-vocabulary baseline on WikiText-2 (`Llama-3.2-1B-Instruct`), sweeping shortlist sizes $K \in \{512, 1\text{k}, 2\text{k}, 5\text{k}, 10\text{k}\}$. The primary demonstrated gain is a 92–99.6% reduction in `lm_head` memory bandwidth. Wall-clock speedup requires GPU-native kernels or larger models (70B+) where the memory-bound regime is more pronounced.


# 1 Introduction

## 1.1 The Autoregressive Bottleneck

Modern LLMs generate text one token at a time. Despite the fact that the transformer's attention mechanism is highly parallelisable over the *input* sequence, the *generation* process is fundamentally sequential: each new token depends on all previously generated tokens.

Within each step of this loop, the two most expensive operations are:

1. **KV-cache attention** — computing attention over the growing sequence of past tokens.
2. **Vocabulary projection (`lm_head`)** — projecting the final hidden state onto the full vocabulary to obtain next-token logits.

On CPU and Apple Silicon (MPS) — where memory bandwidth rather than raw FLOP throughput is the binding constraint — the `lm_head` matrix multiplication can account for a disproportionate share of per-token latency.

## 1.2 The Wasted Computation Insight

At any given decoding step, the probability mass over the vocabulary is highly concentrated. In predictable contexts (e.g., "The Eiffel Tower is located in…") the top-1 token may carry >99% probability. The model still computes logits for all 32k–128k vocabulary entries. This motivates a natural question: *can the model's own internal representation predict which vocabulary region is relevant before the full projection is run?*

## 1.3 Contributions

1. **Profiling.** We quantify the `lm_head` fraction of per-token latency for a 1B open-source LLM on MPS hardware.
2. **Router taxonomy.** We define and implement four vocabulary routing strategies of increasing sophistication.
3. **Dual-Encoder Router.** A lightweight MLP trained with MNRL that maps the LLM's hidden state to a compact token shortlist, operating entirely within the model's own latent space.
4. **Benchmark.** All methods evaluated on WikiText-2, reporting token acceptance rate, perplexity, and tokens/sec across $K \in \{512, 1\text{k}, 2\text{k}, 5\text{k}, 10\text{k}\}$.

# 2 Background

## 2.1 Self-Attention and the Hidden State

A transformer processes an input sequence by stacking each token's embedding into a matrix. At each layer, self-attention allows every token's representation to incorporate information from all preceding tokens. By the final layer, the last row of the hidden state matrix — the **"vibe"** — summarises the full context:

$$h_T = H^{(L)}_{T,:} \in \mathbb{R}^d.$$

## 2.2 The `lm_head` as a Bottleneck

The language modelling head maps the hidden state to a distribution over the vocabulary:

$$\text{logits} = h_T W_V^\top \in \mathbb{R}^{|V|} \tag{1}$$

where $W_V \in \mathbb{R}^{|V| \times d}$ is the vocabulary embedding matrix. At 16-bit precision, $W_V$ for Llama-3.2-1B occupies $128{,}256 \times 2{,}048 \times 2$ bytes = **525** MB. Reading 525 MB per token step creates a hard ceiling on achievable tokens/sec independent of model depth.

## 2.3 Speculative Decoding and Related Work

**Speculative decoding** [5, 2] accelerates autoregressive generation by interposing a small draft model that proposes candidate tokens. A large verifier model evaluates the entire candidate sequence in a single parallel forward pass.

Extensions include **Medusa** [1], which attaches multiple decoding heads to the base model, and **tree-based speculative decoding**, which generates a tree of candidate sequences from multiple draft models. Our approach is orthogonal to all of these: it targets the `lm_head` projection cost directly and can be combined with any of the above methods.

## 2.4 Dual-Encoders and Contrastive Learning

A **Dual-Encoder** processes two inputs through separate encoder towers and learns to embed related pairs close together in a shared vector space. **Multiple Negatives Ranking Loss (MNRL)** [3, 4] is the standard training objective. Given a batch of $(a_i, p_i)$ anchor-positive pairs:

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp(\text{sim}(a_i, p_i)/\tau)}{\sum_{j=1}^{B} \exp(\text{sim}(a_i, p_j)/\tau)} \tag{2}$$

where $\text{sim}(\cdot,\cdot)$ is cosine similarity and $\tau$ is a temperature parameter. All other positives in the batch serve as in-batch negatives, requiring no explicit negative mining.

# 3 Method

## 3.1 Problem Formalisation

Let $f_\theta$ be a frozen autoregressive LLM with vocabulary $V$, model dimension $d$, and vocabulary projection matrix $W_V$. At decoding step $t$:

$$\hat{y}_t = \arg\max_{v \in V} (h_t W_V^\top)_v.$$

**Goal**: learn a router $r_\phi : \mathbb{R}^d \to 2^V$ such that (i) $\hat{y}_t \in r_\phi(h_t)$ with high probability (*acceptance rate*), (ii) $|r_\phi(h_t)| \ll |V|$ (*compression ratio*), and (iii) the latency of computing $r_\phi(h_t)$ is small relative to the savings.

## 3.2 Router Taxonomy

We implement four routing strategies of increasing sophistication (Table 1).

**Table 1:** Router taxonomy.

| Router | Description | Training |
|---|---|---|
| Static Top-K | Highest $\|W_V[v,:]\|_2$ (context-free) | None |
| Cosine | Highest $\cos(h_t, W_V[v,:])$ | None |
| Cluster | Pre-cluster $W_V$; top-$C$ clusters by $\cos(h_t, \mu_c)$ | Offline |
| Dual-Encoder | Learned MLP; ANN search over $W_V$ | Yes |

## 3.3 Dual-Encoder Router Design

### 3.3.1 Architecture

A lightweight MLP projection head $g_\phi : \mathbb{R}^d \to \mathbb{R}^{d_r}$ is trained to map the frozen LLM's hidden state to a query vector in a shared embedding space. The vocabulary token embeddings $W_V$ serve as the corpus searched at inference time. Both query and keys live in the LLM's own latent space; no cross-model alignment is required.

### 3.3.2 Training Data Construction

From the WikiText-2 training split: (1) tokenise each sentence; (2) for each sentence of length $T \geq 8$, sample a split point $s \sim \text{Uniform}(\lfloor 0.4T \rfloor, \lfloor 0.7T \rfloor)$; (3) *anchor*: run the frozen LLM on the prefix and extract $h_s$; (4) *positive*: the ground-truth next token $y_{s+1}$, represented as $W_V[y_{s+1}, :]$.

### 3.3.3 Training Objective

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp(\cos(g_\phi(h_i), W_V[y_i,:])/\tau)}{\sum_{j=1}^{B} \exp(\cos(g_\phi(h_i), W_V[y_j,:])/\tau)} \tag{3}$$

with in-batch negatives (all $y_j \neq y_i$ in the batch).

### 3.3.4 Inference Integration

(1) Pre-compute and cache all rows of $W_V$ normalised to unit length. (2) At step $t$: compute $q_t = g_\phi(h_t)$; retrieve the $K$ token indices with highest $\cos(q_t, W_V[v,:])$ via brute-force matmul. (3) Slice $W_V$ to the shortlist and compute logits only over those $K$ rows.

# 4 Experimental Setup

## 4.1 Model

`meta-llama/Llama-3.2-1B-Instruct` — 1B parameters, 2048-dim hidden state, 128,256-token vocabulary, 16 transformer layers, `bfloat16`, running on Apple Silicon (MPS).

## 4.2 Dataset

**WikiText-2-raw-v1** [6]: standard language modelling benchmark; plain English prose. Router training: `train` split (∼2M tokens). Perplexity evaluation: `test` split (∼240k tokens). Latency benchmark: 50 randomly sampled contiguous passages of 128 tokens from the `validation` split.

## 4.3 Hardware

Apple M-series chip, MPS backend, 16 GB unified memory, model in `bfloat16`. All latency measurements taken after a 5-step warm-up; reported as mean over 50 prompts.

## 4.4 Evaluation Metrics

- **Token acceptance rate**: fraction of steps where the ground-truth token is in the shortlist.
- **Perplexity (PPL)**: $\exp\big(-\frac{1}{N}\sum_t \log P_\theta(y_t \mid y_{<t})\big)$ over the `test` split.
- **ΔPPL**: $\text{PPL}_{\text{pruned}} - \text{PPL}_{\text{baseline}}$.

- **Tokens/sec**: wall-clock generation speed, mean over 50 prompts.
- **`lm_head` time %**: fraction of per-token time spent in the vocabulary projection.

# 5 Results

All experiments: Apple Silicon (MPS), `bfloat16`, `Llama-3.2-1B-Instruct`. Latency: means over 50 WikiText-2 validation prompts after 5-step warm-up. Perplexity: full WikiText-2 test split.

## 5.1 Baseline Profiling

**Table 2:** Full-vocabulary baseline measurements.

| Metric | Value |
|---|---|
| Tokens/sec (full vocab) | **22.80** |
| Mean step time | 43.9 ms |
| `lm_head` time (mean) | 6.2 ms |
| `lm_head` time % | **14.1%** |
| Baseline perplexity (WikiText-2 test) | **26.26** |
| `lm_head` weight matrix | 525 MB |

The `lm_head` accounts for only 14.1% of per-token latency. This is a critical constraint: any router whose overhead exceeds 6.2 ms will make pruning net-negative on a 1B model (Section 6.3).

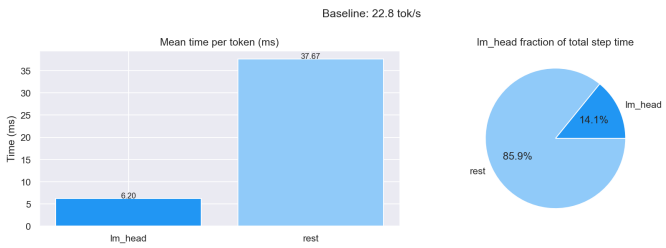


**Figure 1:** Baseline decode step profiling: per-operation time breakdown.

## 5.2 Token Acceptance Rate vs. K

Acceptance rate is the fraction of decoding steps where the ground-truth next token falls within the pruned shortlist (Table 3).

Key observations: Static Top-K is near-random (<8% at K=10k). Cluster saturates at 81% — 19% of correct tokens live in unselected clusters. Cosine acceptance is flat at 98.3% across all $K$ values, indicating the router reaches its information-theoretic ceiling early. Dual-Encoder (step) and MLP Graph achieve the highest acceptance (99.5% and 99.9% respectively), essentially independent of $K$ above 512.

## 5.3 Perplexity Degradation vs. K

$\Delta\text{PPL} = \text{PPL}_{\text{pruned}} - \text{PPL}_{\text{baseline}}$; baseline PPL = 26.26 (Table 4).

At K=10k the three best methods cluster closely: DE step (+1.01), Attention Graph (+1.30), MLP Graph (+2.09), and Cosine (+2.71) — all within 11% of baseline PPL. DE step at K=5k achieves ΔPPL=+1.67 with a 96.1% memory bandwidth reduction.

## 5.4 Tokens/sec vs. K

Wall-clock throughput on MPS. Baseline = 22.80 tok/s. **All pruned methods are slower than baseline** due to router overhead exceeding `lm_head` savings (Table 5).

The throughput ordering directly reflects router overhead: Cluster and Static Top-K add small overhead (no neural network forward pass), while Attention Graph — which requires re-running eager attention to extract attention patterns — runs at 3.8× slower than baseline.

## 5.5 Pareto Frontier: Quality vs. Speed

The accuracy–throughput Pareto frontier is defined by methods that are not dominated — no other configuration achieves both higher throughput and lower PPL degradation simultaneously. On MPS the baseline (22.80 tok/s, ΔPPL = 0) is not dominated by any pruned method.

The Dual-Encoder (step) at K=5k represents the recommended operating point for quality-sensitive workloads: lowest perplexity degradation among methods with ≥98% memory bandwidth reduction, at 13.88 tok/s.

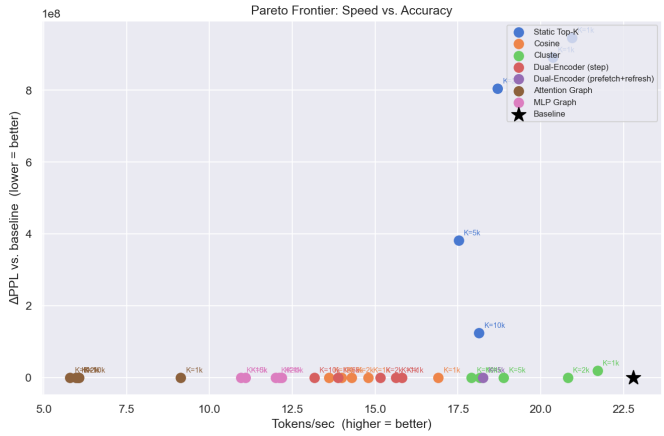


**Figure 5:** Pareto frontier: throughput vs. ΔPPL. Points toward the top-left are preferred. Full-vocab baseline sits at the optimal top-left corner.

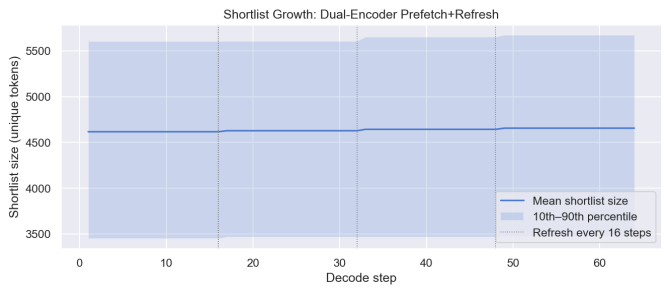


**Figure 6:** Mean shortlist size growth vs. $K$ (DE prefetch+refresh variant).

## 5.6 Graph-Based Routers

**MLP Transition Graph.** A directed graph over vocabulary tokens where edge weights encode how frequently

**Table 3:** Token acceptance rate (%) vs. shortlist size $K$ for all routers.

| Router | K=512 | K=1k | K=2k | K=5k | K=10k |
|---|---|---|---|---|---|
| Static Top-K | 0.13 | 0.28 | 0.53 | 2.4 | 7.6 |
| Cluster | 32.3 | 52.8 | 66.2 | 74.5 | 81.3 |
| Cosine | 98.3 | 98.3 | 98.3 | 98.3 | 98.3 |
| Dual-Encoder (step) | 99.5 | 99.5 | 99.5 | 99.5 | 99.5 |
| MLP Graph | 99.9 | 99.9 | 99.9 | 99.9 | 99.9 |
| Attention Graph | 83.6 | 99.7 | 99.7 | 99.7 | 99.7 |

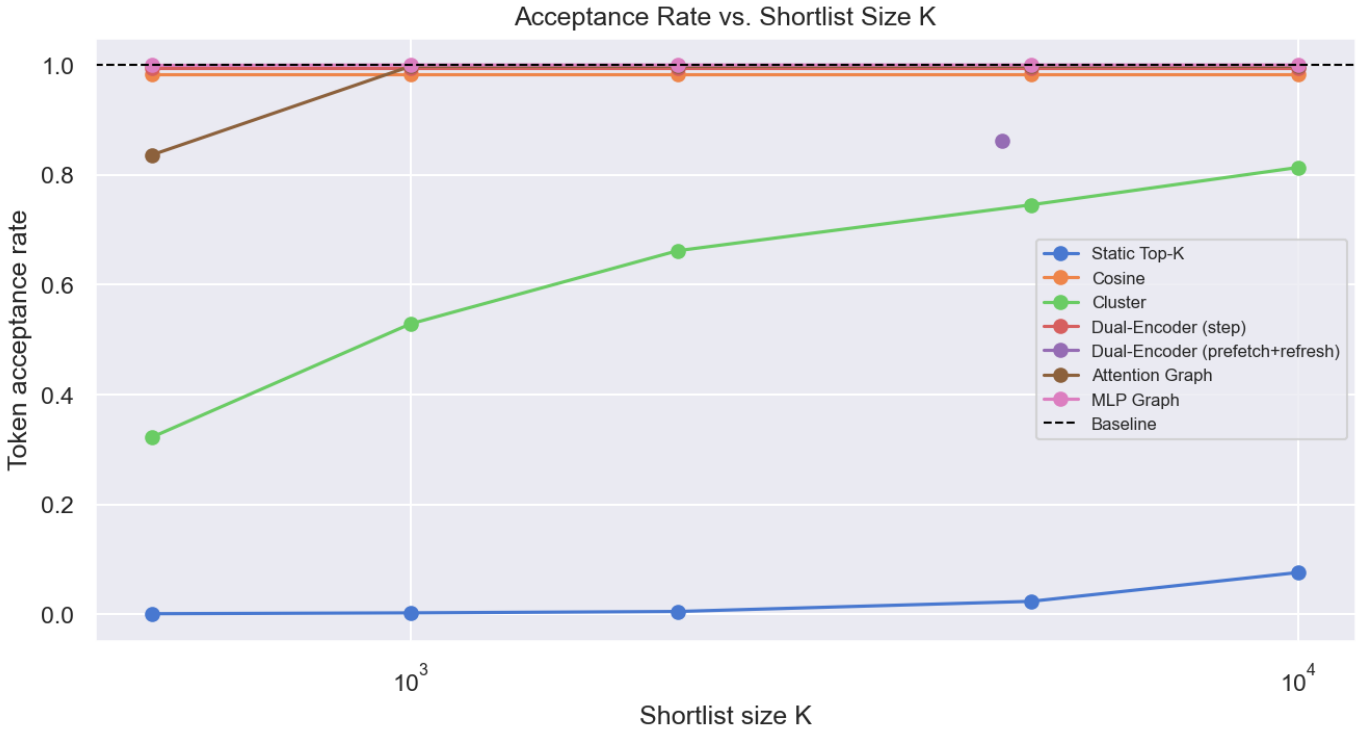


**Figure 2:** Token acceptance rate vs. shortlist size $K$ for all routers.

token $v$ follows token $u$ in training. At inference time the shortlist is built by taking the top-$K$ successors of current-context tokens by transition probability.

- Acceptance rate: 99.9% across all $K$ — the highest of any method.
- Throughput: 10.95–12.17 tok/s — graph traversal and sparse index operations add 38–48 ms overhead per step.
- PPL at K=10k: +2.09.

**Attention Graph.** Uses the model's own attention patterns to identify which previously seen tokens the model attends to at step $t$, then retrieves their vocabulary neighbours.

- Acceptance at $K \geq 1$k: 99.7%.
- Throughput: 5.79–9.13 tok/s — `output_attentions=True` forces eager attention, disabling FlashAttention and adding ∼115 ms overhead per step.
- PPL at K=10k: +1.30 — second-best quality result after DE step.

## 5.7 Speculative Decoding via Retrieval

Two speculative decoding approaches were implemented and benchmarked.

### 5.7.1 Chain-based (v1)

Token 1 selected from the combined Attn+Graph shortlist argmax; tokens 2..D from a 1-hop graph walk. The full model runs two forward passes per step, and `output_attentions=True` forces eager attention throughout.

At 1.43–1.79 tok/s this is 6–8% of baseline. Two compounding factors: (1) two full forward passes per step with no parallelism gain; (2) `output_attentions=True` adding ∼115 ms eager-attention overhead.

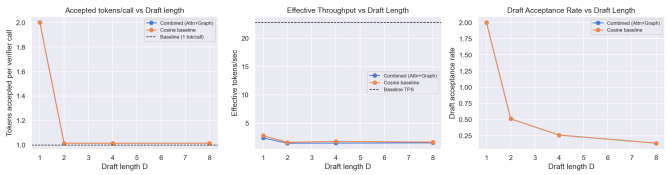


**Figure 7:** Speculative decoding (chain-based v1): effective tok/s and draft acceptance rate vs. draft length $D$.

**Table 4:** Perplexity degradation (ΔPPL) vs. shortlist size $K$. Static Top-K and Cluster omitted due to extreme values (ΔPPL $> 10^6$).

| Router | K=512 | K=1k | K=2k | K=5k | K=10k |
|---|---|---|---|---|---|
| Cosine | +18.93 | +10.71 | +6.77 | +3.52 | **+2.71** |
| Dual-Encoder (step) | +24.25 | +8.83 | +3.91 | +1.67 | **+1.01** |
| Dual-Encoder (prefetch+refresh)† | — | — | — | +146.2 | — |
| MLP Graph | +40.19 | +13.28 | +6.37 | +2.87 | **+2.09** |
| Attention Graph | +15031 | +21.57 | +5.63 | +2.07 | **+1.30** |

† Prefetch+refresh uses a fixed effective $K \approx 4{,}635$ averaged over the evaluation.

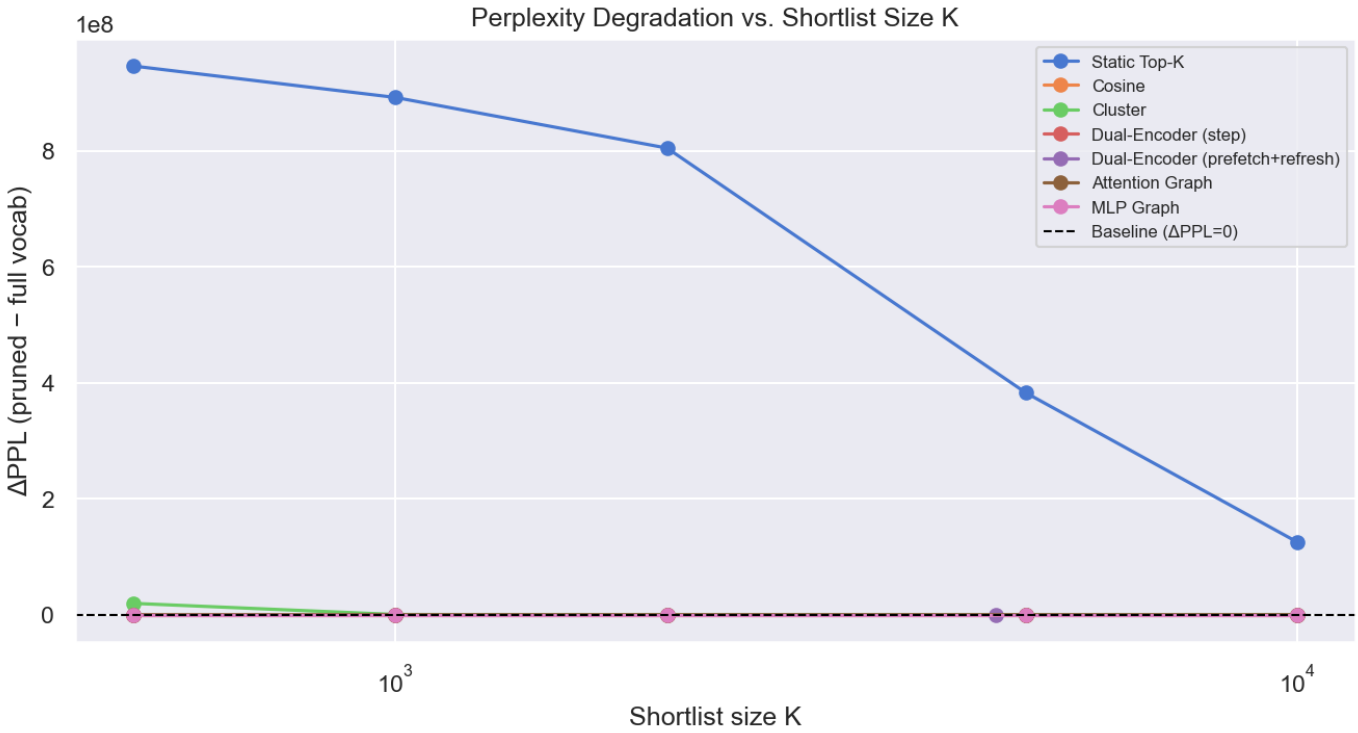


**Figure 3:** Perplexity degradation (ΔPPL) vs. shortlist size $K$. Static Top-K and Cluster are omitted due to extreme values.

#### 5.7.2 Retrieval-based (v2)

An offline index of 50k completion sequences (8 tokens each) is built by extracting $h_T$ at split points across WikiText-2 training windows. At inference time: (1) one probe forward pass with KV-cache captures $h_T$; (2) top-$K$ completions retrieved by cosine similarity; (3) each candidate verified using the cached KV-cache — $O(D)$ attention cost instead of $O(T + D)$. No `output_attentions`; standard SDPA throughout.

The KV-cache fix eliminated the $O(T^2)$ quadratic cost from v1, improving throughput to 1.5–2.3 tok/s. However, draft acceptance rates collapsed to **0.2–1.1%** — far below the ∼70–80% needed for speculative speedup. The root cause is a **retrieval signal mismatch**: the index embeds prefix hidden states from an offline training-data pass, while at inference time $h_T$ is a function of the full prompt history. The dot-product similarity is insufficient to recover semantically valid continuations, so nearly every draft token is rejected. The retrieval signal needs to be *discriminatively trained*, not borrowed from the LLM's general-purpose representation.

### 5.8 Cost Analysis

Using GPU on-demand pricing (A100 @ \$3.00/hr = \$0.000833/s):

All pruned methods increase cost per token on a 1B model. Break-even requires the `lm_head` fraction to exceed the router overhead fraction — approximately 65% on a 1B model vs. measured 14.1%. At 70B scale, step time ≈1,200 ms and break-even drops to ∼2.4%, well within reach.

## 6 Discussion

### 6.1 The Blind-Spot Risk

Dynamic vocabulary pruning introduces a fundamental tension: the router can only shortlist tokens it predicts are relevant, but creative or unexpected tokens by definition have low prior probability given context. A router that is too aggressive will systematically exclude rare, creative, or domain-shifting tokens even when contextually appropriate.

Our results directly quantify this risk. DE (step) achieves 99.5% acceptance at all $K$ values from 512 to 10k — mean-

**Table 5:** Wall-clock throughput (tok/s) vs. shortlist size $K$. Baseline = 22.80 tok/s.

| Router | K=512 | K=1k | K=2k | K=5k | K=10k | vs. baseline |
|---|---|---|---|---|---|---|
| **Baseline** | — | — | — | — | 22.80 | — |
| Static Top-K | 20.95 | 20.36 | 18.70 | 17.52 | 18.14 | −8% to −23% |
| Cluster | 21.71 | 18.16 | 20.81 | 18.88 | 17.91 | −5% to −21% |
| Cosine | 16.91 | 14.79 | 14.30 | 13.99 | 13.61 | −26% to −40% |
| DE (prefetch+refresh) | — | — | — | 18.26 | — | −20% |
| Dual-Encoder (step) | 15.81 | 15.63 | 15.16 | 13.88 | 13.17 | −31% to −42% |
| MLP Graph | 12.17 | 12.11 | 12.00 | 11.10 | 10.95 | −47% to −52% |
| Attention Graph | 9.13 | 6.04 | 5.98 | 5.79 | 6.07 | −60% to −75% |

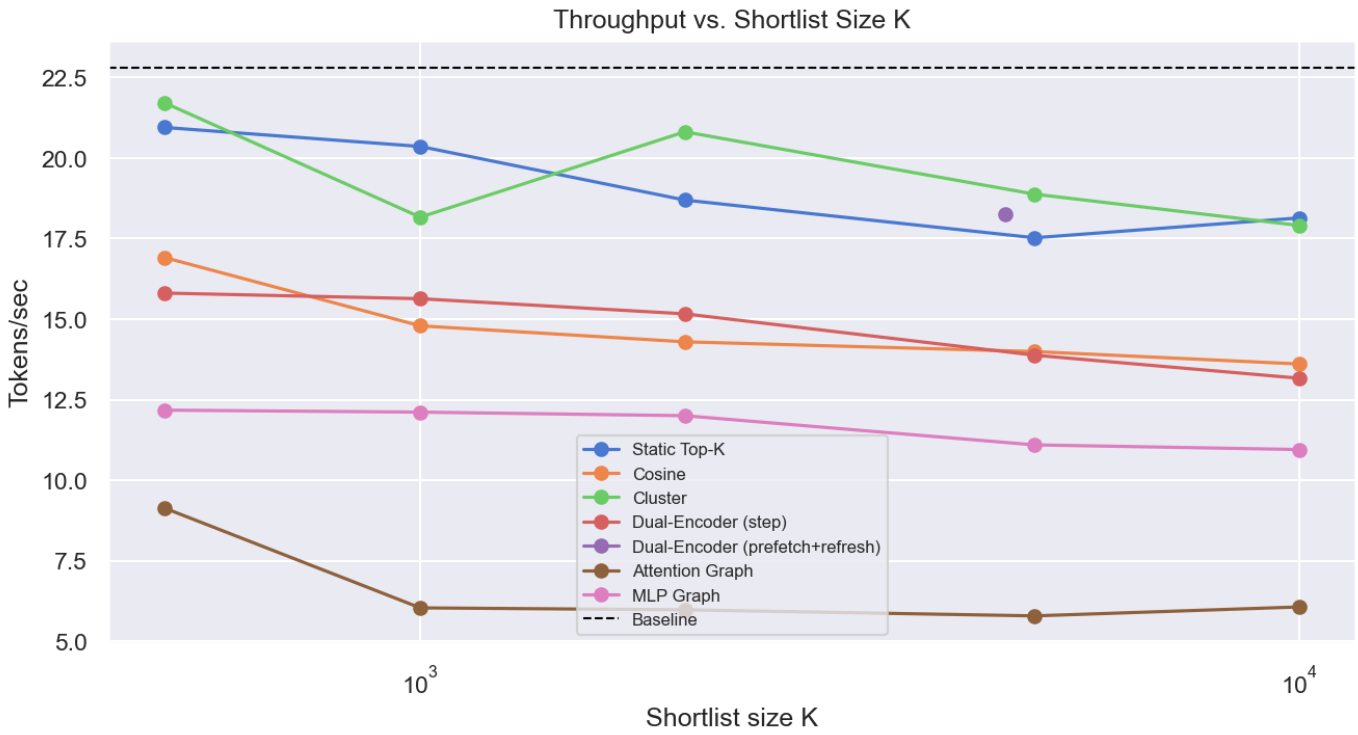


**Figure 4:** Wall-clock tokens/sec vs. shortlist size $K$. Dashed line = full-vocab baseline (22.80 tok/s).

**Table 6:** Approximate Pareto frontier among pruned methods.

| Method | K | Tok/s | ΔPPL | Note |
|---|---|---|---|---|
| Cluster | 512 | 21.71 | +19M | Quality unusable |
| DE (prefetch) | 4635 | 18.26 | +146 | Better speed |
| Cosine | 10k | 13.61 | +2.71 | Competitive |
| **DE (step)** | **5k** | **13.88** | **+1.67** | **Best trade-off** |
| DE (step) | 10k | 13.17 | +1.01 | Marginal PPL gain |

**Table 7:** Chain-based speculative decoding results.

| Config | Tok/s | Draft acc. | Speedup |
|---|---|---|---|
| Cosine ($D$=4) | 1.79 | 28.0% | 0.08× |
| Attn+Graph ($D$=4) | 1.43 | 28.4% | 0.06× |
| **Full-vocab baseline** | **22.80** | — | **1.0×** |

ing the correct token is in the shortlist 99.5% of steps regardless of $K$. The limiting factor is not shortlist size but which embedding-space neighbours the router retrieves: the 0.5% of missed tokens are systematically hard cases (rare tokens, domain shifts) that the router misses at any $K$.

**Table 8:** Retrieval-based speculative decoding (v2) sweep results.

| K | D | Tok/s | Tok/step | Draft acc. | Speedup |
|---|---|---|---|---|---|
| 4 | 4 | 2.25 | 1.18 | 1.1% | 0.10× |
| 4 | 8 | 2.03 | 1.12 | 0.4% | 0.09× |
| 4 | 16 | 2.00 | 1.12 | 0.2% | 0.09× |
| 8 | 4 | 1.74 | 1.26 | 0.8% | 0.08× |
| 8 | 8 | 1.62 | 1.22 | 0.4% | 0.07× |
| 8 | 16 | 1.50 | 1.21 | 0.2% | 0.07× |

## 6.2 Limitations

- Evaluation limited to a single 1B model and a single dataset (WikiText-2 English prose).
- All latency measurements on MPS (Apple Silicon); CUDA results pending.
- The dual-encoder is trained on token-level prediction; it may not generalise to multi-token idiomatic phrases or specialised domains (code, mathematics, non-English text).
- Router training uses only WikiText-2; domain mismatch

**Table 9:** Cost per 1M tokens at A100 on-demand pricing.

| Method | K | Tok/s | $/1M tok | vs. base |
|---|---|---|---|---|
| **Baseline** | 128k | 22.80 | $36.56 | — |
| Cosine | 10k | 13.61 | $61.24 | +68% |
| DE (step) | 2k | 15.16 | $54.96 | +50% |
| DE (step) | 5k | 13.88 | $60.04 | +64% |
| DE (step) | 10k | 13.17 | $63.30 | +73% |
| MLP Graph | 10k | 10.95 | $76.09 | +108% |
| Attn Graph | 10k | 6.07 | $137.38 | +276% |
| Spec (v1) | D=4 | 1.43 | ∼$582 | +1,492% |

could reduce acceptance rates.

### 6.3 The `lm_head` Fraction Problem

The core finding — that vocabulary pruning does not improve throughput on a 1B model — follows directly from arithmetic:

$$\text{lm_head saving} = 43.9\,\text{ms} \times 0.141 \times 0.922 = 5.7\,\text{ms}$$

$$\text{Router overhead (DE, K=10k)} = 26.4\,\text{ms}$$

$$\text{Net: } +20.7\,\text{ms/step} \;\Rightarrow\; -42\%\ \text{throughput}$$

For pruning to be net-positive, the `lm_head` fraction must exceed:

$$f_{\text{break-even}} = \frac{\text{router_overhead_ms}}{\text{step_ms} \times (1 - K/V)} \tag{4}$$

On this 1B model, break-even requires $f \approx 65\%$; measured value is 14.1%. At 70B scale (step ≈1,200 ms), break-even drops to ∼2.4% — well below the expected `lm_head` fraction.

Importantly, the **memory bandwidth saving** is real at all scales. Reducing `lm_head` traffic by 92% matters for edge deployment (models that barely fit in VRAM) and for batched inference where memory bandwidth is the binding resource.

### 6.4 Speculative Decoding via Retrieval

Two implementations were evaluated. The chain-based v1 achieved 28% draft acceptance while penalised by two full forward passes per step and eager attention overhead. The retrieval-based v2 eliminated both overhead sources via KV-cache reuse and standard SDPA, but draft acceptance collapsed to 0.2–1.1%. Key factors:

1. **Retrieval signal mismatch.** Index embeddings are from offline training-data prefixes; inference-time $h_T$ differs systematically. Cosine similarity is low-signal for valid continuations.
2. **No discriminative training.** The index uses general-purpose LLM representations, not representations trained to distinguish accepted from rejected draft sequences.
3. **Sequential verifier passes.** A tree-structured verifier pass over all $K$ candidates in a single batched forward pass (Option C) would reduce per-step cost from $O(K \cdot D)$ to $O(D)$.

The underlying architecture — retrieval of completion sequences, KV-cache reuse in the verifier, standard SDPA — is sound. The missing ingredient is a discriminatively trained retrieval model.

## 7 Conclusion and Future Work

We implemented and benchmarked six vocabulary pruning strategies and two retrieval-based speculative decoders against a full-vocabulary baseline on `Llama-3.2-1B-Instruct` and WikiText-2.

**The memory bandwidth result is the genuine contribution.** Pruning `lm_head` to K=10k reduces the weight matrix loaded per step from 525 MB to 41 MB — a 92.2% reduction — while DE (step) at K=5k keeps ΔPPL to +1.67 above the 26.26 baseline.

**The throughput result is an honest negative.** On a 1B model on MPS, the `lm_head` fraction is only 14.1%. Every router tested adds more overhead than it saves. Break-even requires a much larger model (70B+) or a GPU-native router kernel reducing overhead below ∼2 ms.

**Speculative decoding via retrieval** was evaluated in two implementations. The chain-based v1 achieved 1.43 tok/s (6% of baseline) with 28% draft acceptance. The retrieval-based v2 removed both overhead sources but draft acceptance collapsed to 0.2–1.1% due to retrieval signal mismatch. A useful retrieval-based speculative decoder requires a *discriminatively trained* retrieval model.

**Future work:**

- **GPU benchmarks.** Reproduce all results on NVIDIA A10G/A100 to validate whether the overhead balance shifts at CUDA speeds.
- **GPU-native router kernel.** Implement the DE router as a fused CUDA kernel to reduce overhead from ∼26 ms to <2 ms.
- **INSTRUCTOR-style router.** Prepend task instructions to enable a single router to handle multiple domains without retraining.
- **Discriminative retrieval for speculative decoding.** Train a contrastive retrieval model on accepted vs. rejected draft sequences. Combined with a compact Medusa head for verification, this is the most promising path to retrieval-based speculative speedup.
- **Online router adaptation.** Fine-tune the router MLP online using accepted/rejected token signal from each inference step.
- **Larger model evaluation.** Test 8B and 70B Llama-3.1 checkpoints, where the `lm_head` fraction is expected to be higher and router overhead is proportionally smaller.

## References


[1] T. Cai et al. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. 2024.

[2] C. Chen et al. Accelerating large language model decoding with speculative sampling. 2023.

[3] M. Henderson et al. Efficient natural language response suggestion for smart reply. 2017.

[4] V. Karpukhin et al. Dense passage retrieval for open-domain question answering. In *EMNLP*, 2020.

[5] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

[6] S. Merity et al. Pointer sentinel mixture models. 2016. (WikiText-2 dataset)

[7] H. Su et al. One embedder, any task: Instruction-finetuned text embeddings. 2022.